



Faculty of Engineering and Technology
Electrical and Computer Engineering Department
Artificial Intelligent (AI)

ENCS3340

Project 1

Name	ID	section
Sari Murad Abdalghani	1220982	4
Mohammad Shamasneh	1220092	1

Instructor: Dr. Yazan Abu Farha

Date: 4/5/2025

Abstract

In our project we were use simulated annealing, and genetic algorithms in delivery packages project, we should assign each package to one of the available delivery vehicles that we have and also determine the route that each vehicle will follow to minimize the operational cost, which is defined by the total distance traveled by all vehicles. While the goal is to prioritize delivering higher-priority packages first, this is not a strict constraint and can be violated if doing so would result in significantly higher costs. Each vehicle has a limited capacity (in kg). You need to ensure that the total weight of the packages assigned to a vehicle does not exceed its capacity.

1. Genetic Algorithms (GAs)

Genetic algorithms (GAs) are a type of computational optimization technique inspired by the principles of natural selection and genetics. They are used to solve complex problems by mimicking the process of evolution to improve a population of potential solutions iteratively. These algorithms operate on a set of candidate solutions encoded as strings of binary digits or other data structures.

1.1 Over view of Genetic Algorithms function:

```
Tabnine | Edit | Test | Explain | Document
134 def genetic_algorithm():
135     population_size = 100
136     mutation_rate = 0.1
137     generations = 500
138
139     population = create_population(population_size)
140     best_solution = min(population, key=lambda x: evaluate_solution(x))
141     for generation in range(generations):
142         new_population = []
143
144         for size in range(population_size):
145             parent1 = select_parent(population)
146             parent2 = select_parent(population)
147             child = crossover(parent1, parent2)
148             child = mutate(child, mutation_rate)
149             new_population.append(child)
150
151         population = new_population
152         current_best = min(population, key=lambda x: evaluate_solution(x))
153         if evaluate_solution(current_best) < evaluate_solution(best_solution):
154             best_solution = current_best
155     return best_solution
156
```

Figure 1: Genetic Algorithms function

1.2 Crossover and select parent

```
Tabnine | Edit | Test | Explain | Document
115 def select_parent(population):
116     num = random.randint(0, len(population)-1)
117     parent = population[num]
118     return parent
119
Tabnine | Edit | Test | Explain | Document
120 def crossover(parent1, parent2):
121     point = random.randint(1, len(parent1)-2)
122     child = parent1[:point] + parent2[point:]
123     return child
124
```

Figure 2: Crossover and select parent function

1.3 Mutate function:

```
Tabnine | Edit | Test | Explain | Document
125 def mutate(solution, mutation_rate):
126     if random.random() < mutation_rate:
127         n, m = random.sample(range(len(solution)), 2)
128         temp = solution[n]
129         solution[n] = solution[m]
130         solution[m] = temp
131     return solution
132
```

Figure 3: Mutate function

2. Simulated Annealing

Is a method used to find good solutions to hard problems. It works by trying different answers and sometimes accepting worse ones to help escape bad spots (called local minima). At the beginning, it explores a lot (like when things are "hot"), but over time it becomes more careful (as it "cools down"). This helps it find a better overall solution in the end.

In this project, Simulated Annealing is used to assign packages to delivery cars in a smart way. The goal is to reduce the total travel distance while considering the priority of each package. At first, the algorithm tries many different ways to assign packages. Sometimes it even accepts worse assignments to avoid getting stuck. As the process continues, it becomes more selective, helping it find a better and more balanced distribution of packages across the cars.

2.1. Random Initial State:

```
95 # ----- Random Initial State -----
96
97 def randomInitialState() :
98     global states
99     states = []
100     for car in cars_list :
101
102         point = {
103             "name" : car['name'],
104             "capacity" : car["capacity"],
105             "packages" : [[0,0,0]]
106         }
107         states.append(point)
108
109     packages_list.sort(key=lambda p: p["priority"])
110     for package in packages_list :
111         point = [package["x"],package["y"],package["weight"],package["priority"],package["name"]]
112         test = 100
113         while True :
114             test -=1
115             car = random.choice(states)
116             if package["weight"] <= car["capacity"] :
117                 car["packages"].append(point)
118                 car["capacity"] -= package["weight"]
119                 break
120             elif test == 0 :
121                 print("Cannot distribute package due to capacity limitations.")
122                 print(f'Package {point} could not be assigned.')
123                 test = 100
124                 break
125     return states
126
```

Figure 4: Random Initial State

2.2. Random Next State:

```
127 def randomNextState(states):
128     for i in range(1000):
129         new_states = copy.deepcopy(states)
130         choice = random.random()
131
132         if choice < 0.25:
133             car = random.choice(new_states)
134             if len(car["packages"]) > 2:
135                 point1, point2 = random.sample(range(1, len(car["packages"])), 2)
136                 car["packages"][point1], car["packages"][point2] = car["packages"][point2], car["packages"][point1]
137
138         elif choice < 0.5:
139             car1, car2 = random.sample(new_states, 2)
140             if len(car1["packages"]) <= 1 or len(car2["packages"]) <= 1:
141                 continue
142
143             idx1 = random.randint(1, len(car1["packages"]) - 1)
144             idx2 = random.randint(1, len(car2["packages"]) - 1)
145
146             pkg1 = car1["packages"][idx1]
147             pkg2 = car2["packages"][idx2]
148
149             new_cap1 = car1["capacity"] + pkg1[2] - pkg2[2]
150             new_cap2 = car2["capacity"] + pkg2[2] - pkg1[2]
151
152             if new_cap1 >= 0 and new_cap2 >= 0:
153                 car1["packages"][idx1], car2["packages"][idx2] = pkg2, pkg1
154                 car1["capacity"] = new_cap1
155                 car2["capacity"] = new_cap2
156
157         elif choice < 0.75:
158             car_from, car_to = random.sample(new_states, 2)
159             if len(car_from["packages"]) <= 1:
160                 continue
161
162             idx = random.randint(1, len(car_from["packages"]) - 1)
163             pkg = car_from["packages"][idx]
164
165             if car_to["capacity"] >= pkg[2]:
166                 car_from["packages"].pop(idx)
167                 car_from["capacity"] += pkg[2]
168                 car_to["packages"].append(pkg)
169                 car_to["capacity"] -= pkg[2]
170
171             else:
172                 car = random.choice(new_states)
173                 if len(car["packages"]) <= 2:
174                     continue
175
176                 base = [0, 0]
177                 remaining = car["packages"][1:]
178                 ordered = []
179
180                 while remaining:
181                     nearest = min(remaining, key=lambda p: calculation(base[0], base[1], p[0], p[1]))
182                     ordered.append(nearest)
183                     base = [nearest[0], nearest[1]]
184                     remaining.remove(nearest)
185
186                 car["packages"] = [[0, 0, 0]] + ordered
187
188     return new_states
189
190 return states
191
```

Figure 5: Random Next State part 1

```
172
173     idx = random.randint(1, len(car_from["packages"]) - 1)
174     pkg = car_from["packages"][idx]
175
176     if car_to["capacity"] >= pkg[2]:
177         car_from["packages"].pop(idx)
178         car_from["capacity"] += pkg[2]
179         car_to["packages"].append(pkg)
180         car_to["capacity"] -= pkg[2]
181
182     else:
183         car = random.choice(new_states)
184         if len(car["packages"]) <= 2:
185             continue
186
187         base = [0, 0]
188         remaining = car["packages"][1:]
189         ordered = []
190
191         while remaining:
192             nearest = min(remaining, key=lambda p: calculation(base[0], base[1], p[0], p[1]))
193             ordered.append(nearest)
194             base = [nearest[0], nearest[1]]
195             remaining.remove(nearest)
196
197         car["packages"] = [[0, 0, 0]] + ordered
198
199     return new_states
200
201 return states
202
```

Figure 6 : Random Next State (part 2)

2.3. Simulated Annealing:

```
311 def simulatedAnnealing(T_initial=1000, cooling_rate=0.95, max_iter=1000):
312     T = T_initial
313     current = randomInitialState()
314
315     for i in range(max_iter):
316         T *= cooling_rate
317         if T < 1 :
318             break
319
320         nextState = randomNextState(current)
321         delta_E = objectiveFunc(current) - objectiveFunc(nextState)
322         #print(objectiveFunc(nextState))
323         if delta_E > 0:
324             current = nextState
325         elif random.random() < calculationProbability(delta_E, T):
326             current = nextState
327
328     return current
329
```

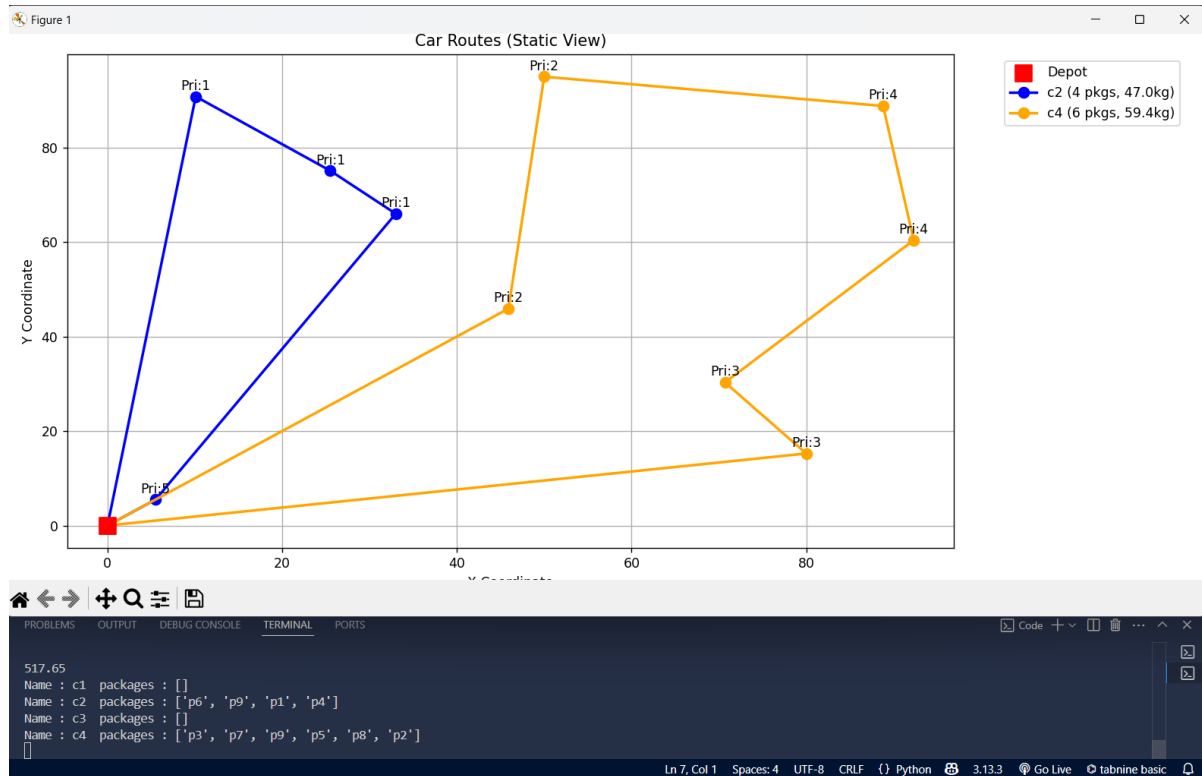
Figure 7: Simulated Annealing

Test case 1:

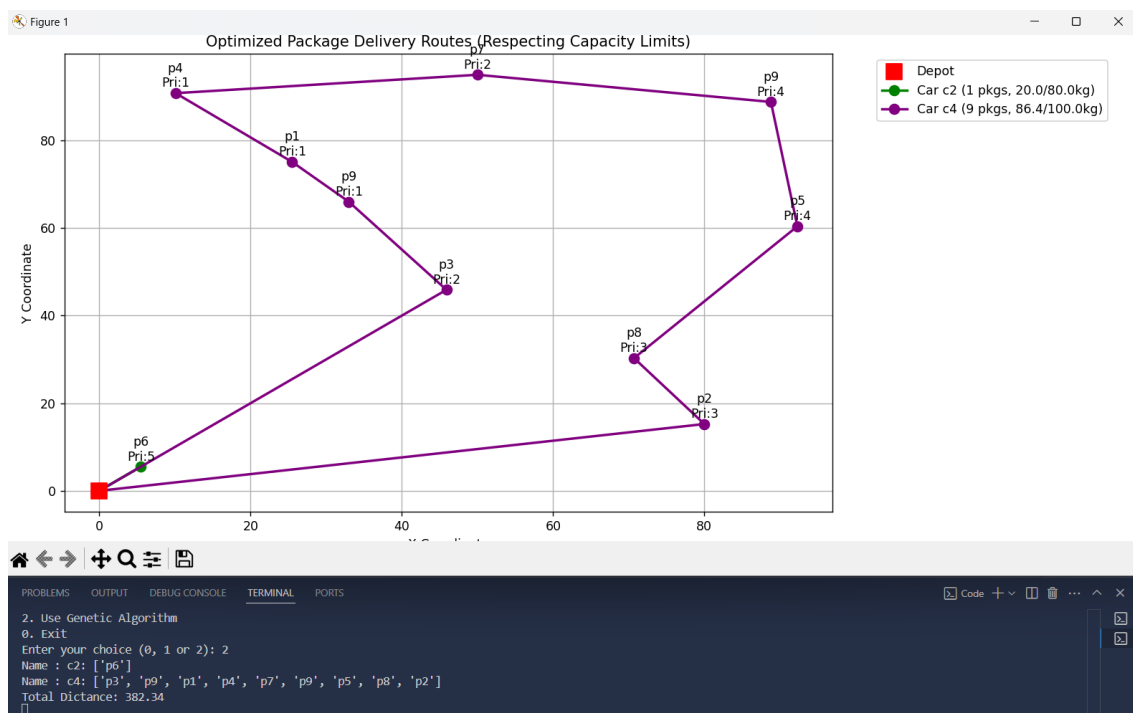
```
≡ cars.txt
1   c1,60
2   c2,80
3   c3,50
4   c4,100
5   
```

```
≡ packages.txt
1   p1,25.5,75.1,10.5,1
2   p2,80,15.25,8,3
3   p3,45.9,45.9,15,2
4   p4,10.1,90.8,5.5,1
5   p5,92.3,60.4,12.8,4
6   p6,5.5,5.5,20,5
7   p7,50,95,7.2,2
8   p8,70.7,30.3,9.9,3
9   p9,33,66,11,1
10  p9,88.8,88.8,6.5,4
```

SA output:



GA output:



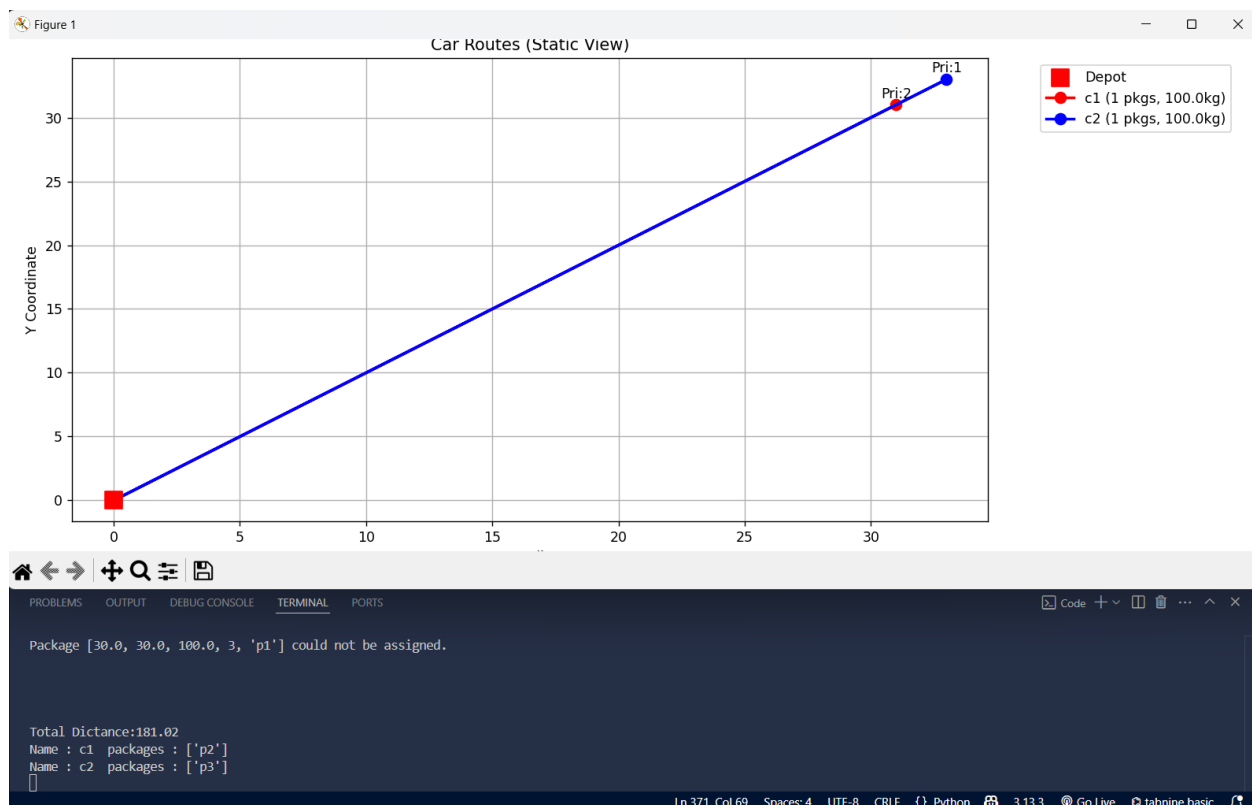
- For first test case we noticed that GA is better than SA because the distance in SA is 517Km and GA is 382 so GA is better.
- In addition, we noticed that GA is slower than SA in running time, that's because SA work randomly and GA do generations and get best.
- For SA if we test it more than one time usually give us new output and may be the output is like GA output

Test case 1:

```
cars.txt
1 c1,100
2 c2,100
3
```

```
packages.txt
1 p1,30,30,100,3
2 p2,31,31,100,2
3 p3,33,33,100,1
```

SA output:



GA output:



- For this case both GA and SA have the same output
- In addition, we noticed that GA is slower than SA in running time, that's because SA work randomly and GA do generations and get best.
- The two algorithms didn't assign to p1 because its priority is 3(bad) and the three packages are behind each other.